



S2-CP02 - Projeto Motiva

Atualizacao Remota de Firmware (OTA)

Relatorio Final

Giovanna Fernandes Pereira - RM 565434

João Pedro de Moura Albino - RM 565323

Kauê Silva Matheus - RM 561675

Professor: Marcelo Fernando Morgantini

São Paulo - Setembro de 2026

1. Identificacao da entrega

Este relatorio documenta a implementacao e a validacao de uma solucao de atualizacao remota de firmware (OTA) para um ESP32 simulado no Wokwi, no contexto do Projeto Motiva. A solucao foi desenvolvida em duas versoes: Firmware 1.0, responsavel pelo monitoramento inicial e pela verificacao de atualizacao, e Firmware 2.0, que acrescenta ordenacao, mediana e histerese.

Projeto Wokwi	https://wokwi.com/projects/475891405739849729
Repositório GitHub	https://github.com/jalbino0/motiva-ota
Firmware 2.0 (.bin)	https://raw.githubusercontent.com/jalbino0/motiva-ota/main/firmware_v2.bin

2. Objetivo e requisitos atendidos

O projeto simula um no IoT de monitoramento de vegetacao. Cada sessao gera cinco leituras pseudoaleatorias entre 10 e 20 cm, com intervalo de 2 segundos entre leituras e nova sessao a cada 48 segundos contados a partir do inicio da sessao anterior.

- Firmware 1.0: identifica a versao no Serial, executa 5 leituras, calcula a media, usa LED azul e inicia nova sessao a cada 48 s.
- OTA: apos 3 sessoes completas, conecta ao repositorio remoto, consulta version.json, compara versoes, baixa firmware_v2.bin, grava a nova imagem e reinicia.
- Firmware 2.0: preserva as funcoes anteriores e adiciona copia ordenada das leituras, mediana e histerese.
- LED RGB: FW 1.0 em azul; FW 2.0 NORMAL em verde; FW 2.0 ALERTA em vermelho.

3. Arquitetura da solucao

ESP32 / Wokwi FW 1.0	->	Internet HTTPS	->	GitHub version.json + firmware_v2.bin
-------------------------	----	-------------------	----	---

Fluxo OTA: consultar versao -> comparar versao instalada e remota -> baixar o binario -> gravar na particao OTA inativa -> selecionar nova imagem -> reiniciar -> executar Firmware 2.0.

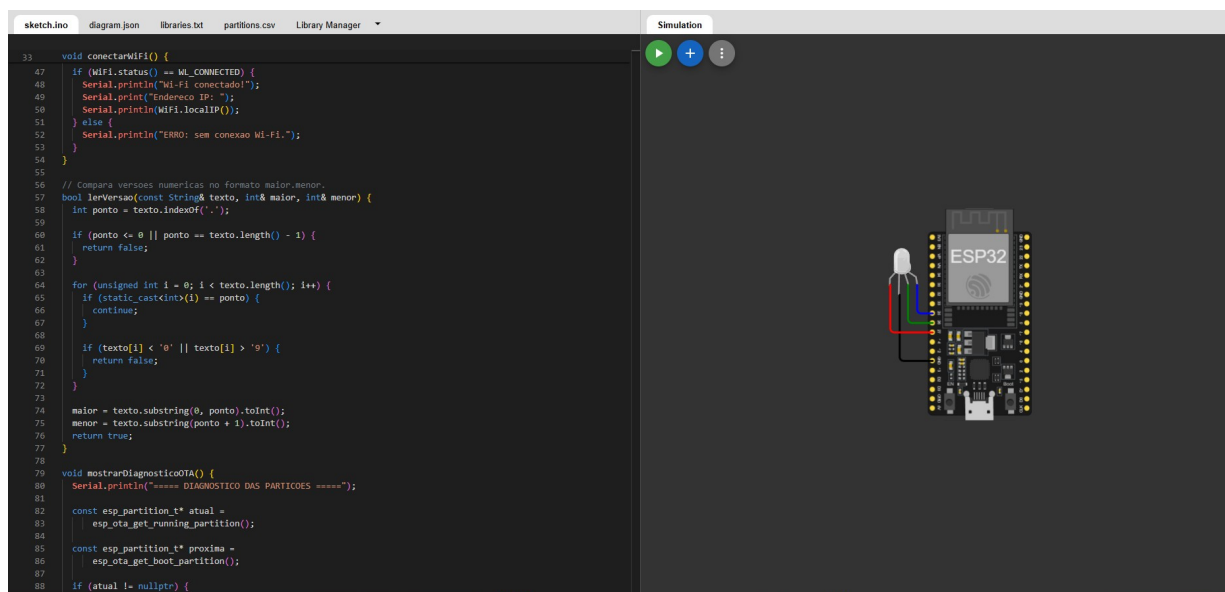


Figura 1 - Circuito do ESP32 com LED RGB no Wokwi.

4. Firmware 1.0

A versao 1.0 realiza a aquisicao das cinco leituras, armazena os valores em vetor, calcula a media e usa o LED azul como indicacao visual. A temporizacao e baseada em millis(), permitindo que as sessoes comecem a cada 48 segundos a partir do inicio da sessao anterior. Apos a terceira sessao concluida, a funcao consultarOTA() acessa o manifesto remoto e inicia a atualizacao quando encontra uma versao superior.

5. Atualizacao OTA

O manifesto remoto informa a versao disponivel e a URL direta do binario. O Firmware 1.0 interpreta o JSON, compara as versoes no formato maior.menor e, quando encontra a versao 2.0, utiliza HTTPUpdate para baixar e gravar a nova imagem.

5.1 Manifesto version.json

```
{
  "version": "2.0",
  "url":
  "https://raw.githubusercontent.com/jalbino0/motiva-ota/main/firmware_v2.bin"
}
```

5.2 Evidencia da atualizacao

O Serial Monitor registrou o seguinte fluxo durante o teste real no Wokwi:

```
===== CONSULTA OTA APÓS 3 SESSÕES =====
Versao instalada: 1.0
Versao disponivel: 2.0
Atualizacao disponivel!
Baixando: https://raw.githubusercontent.com/jalbino0/motiva-ota/main/firmware_v2.bin
OTA concluida com sucesso!
===== DIAGNOSTICO DAS PARTICOES =====
Particao atual: app0, endereco: 0x10000
Particao selecionada: app1, endereco: 0x150000
Verificacao da imagem: ESP_OK
=====
Reiniciando para executar o novo firmware...
...
MONITORAMENTO DE VEGETACAO - FW 2.0
```

O diagnostico confirmou a execucao original em app0, a selecao de app1 para a nova imagem e a verificacao ESP_OK. Apos o reboot por software, o equipamento iniciou exibindo "MONITORAMENTO DE VEGETACAO - FW 2.0".

6. Firmware 2.0

A versao 2.0 mantem cinco leituras por sessao e o calculo da media. Como evolucao funcional, cria uma copia do vetor, ordena os cinco valores em ordem crescente, utiliza o terceiro elemento como mediana e aplica uma histerese baseada nessa mediana. O vetor original e preservado e tambem exibido no Serial Monitor.

6.1 Histerese e LED

Condição da mediana	Estado resultante	LED
≥ 16 cm	ALERTA	Vermelho
> 14 e < 16 cm	Mantem estado anterior	Mantem cor do estado
≤ 14 cm	NORMAL	Verde

6.2 Evidencia de processamento

```
Leitura 1: 18 cm
Leitura 2: 12 cm
Leitura 3: 17 cm
Leitura 4: 20 cm
Leitura 5: 15 cm
Ordem original: 18 12 17 20 15
Ordem crescente: 12 15 17 18 20
Media: 16.4 cm
Mediana: 17 cm
Estado: ALERTA
```

A amostra acima comprova a exibicao das leituras na ordem original, a ordenacao crescente, a media, a mediana e a transicao para ALERTA quando a mediana e maior ou igual a 16 cm.

6.3 Evidencia controlada da histerese

Para comprovar de forma deterministica os tres ramos da histerese, foi executado um teste temporario com medianas controladas 16, 15 e 14. A logica de decisao do firmware nao foi alterada; somente os valores de entrada foram fixados durante a validacao.

```
SESSAO 1
Mediana: 16 cm
Estado: ALERTA

SESSAO 2
Mediana: 15 cm
Estado: ALERTA

SESSAO 3
Mediana: 14 cm
Estado: NORMAL

===== TESTE DE HISTERESE CONCLUIDO =====
Esperado: 16 -> ALERTA, 15 -> ALERTA, 14 -> NORMAL
```

O resultado demonstra que a mediana 16 ativa ALERTA, a mediana 15 preserva ALERTA e a mediana 14 retorna ao estado NORMAL.

7. Temporizacao das sessoes

Nos testes do Firmware 1.0, os inicios das sessoes foram registrados em 4.397 ms, 52.397 ms e 100.397 ms, correspondendo a intervalos exatos de 48.000 ms. No teste controlado do Firmware 2.0, os inicios ocorreram em 4.676 ms, 52.676 ms e 100.676 ms, novamente com diferenca exata de 48.000 ms.

8. Resultados dos testes obrigatorios

Teste	Condição	Resultado observado	Status
1	Firmware 1.0	5 leituras, media e LED azul implementados	OK
2	Sessao completa	Nova sessao apos 48.000 ms	OK
3	Manifesto indica 2.0	Atualizacao identificada apos 3 sessoes	OK
4	OTA executada	Reboot e inicio no Firmware 2.0	OK
5	Firmware 2.0	Media, ordenacao e	OK

		mediana exibidas	
6	Mediana >= 16	Estado ALERTA; LED vermelho	OK
7	Mediana entre 14 e 16	Mediana 15 manteve estado ALERTA	OK
8	Mediana <= 14	Estado NORMAL; LED verde	OK

9. Tratamento de erros e comprovacao dos testes

Este criterio foi tratado em duas frentes: (1) mensagens claras para os cenarios minimos de falha previstos no enunciado e (2) comprovacao dos oito testes obrigatorios, apresentada na Secao 8 e nas evidencias de Serial das Secoes 5, 6 e 7. O Firmware 1.0 tambem trata manifesto JSON invalido, campos obrigatorios ausentes e URL de firmware fora do repositorio esperado.

9.1 Matriz de tratamento minimo de falhas

A tabela abaixo relaciona cada situacao minima pedida no enunciado com a resposta implementada no firmware. Os cenarios de falha nao precisam ocorrer durante a execucao normal; o importante e que o programa os detecte e informe claramente no Serial Monitor.

Situacao prevista	Tratamento implementado	Mensagem / evidência no código
Sem conexao Wi-Fi	Tenta conectar por tempo limitado; se permanecer desconectado, cancela a consulta OTA.	"ERRO: sem conexao Wi-Fi." e "OTA cancelada: sem conexao Wi-Fi."
Manifesto nao pode ser acessado	Verifica se a requisicao HTTPS pode ser iniciada e se o HTTP GET retorna 200.	"ERRO: nao foi possivel iniciar a consulta." ou "ERRO ao acessar manifesto. Código: ..."
Versao instalada ja e a mais recente	Compara versao local e remota; se nao houver versao superior, encerra sem gravar firmware.	"Firmware ja atualizado: nenhuma versao mais nova."
Arquivo de firmware nao pode ser baixado	O HTTPUpdate retorna falha; o firmware imprime código e descricao do erro.	"ERRO no download ou na gravacao OTA: <código> - <descricao>"
Processo de atualizacao retorna erro	O mesmo retorno de erro do HTTPUpdate impede o reboot como se a atualizacao tivesse sido concluida.	Código e mensagem obtidos por getLastError() e getLastErrorString().

9.2 Comprovacao dos testes obrigatorios

Os oito testes obrigatorios foram consolidados na tabela da Secao 8. As evidencias objetivas incluem: intervalos de 48.000 ms entre inicios de sessao; identificacao da versao 2.0 apos tres sessoes; download e gravacao OTA com verificacao ESP_OK; reboot executando o Firmware 2.0; exibicao de media, ordenacao e mediana; e teste controlado de histerese 16 -> ALERTA, 15 -> ALERTA (estado anterior mantido) e 14 -> NORMAL. A indicacao de LED correspondente e definida diretamente pela logica do firmware: azul no FW 1.0, vermelho em ALERTA e verde em NORMAL.

Observacao de rastreabilidade: a demonstracao registrada comprova os testes funcionais e o fluxo OTA de sucesso. Os cenarios de falha acima foram verificados pela logica e pelas mensagens implementadas no código-fonte; eles podem ser reproduzidos alterando temporariamente a conectividade, a URL do manifesto, a versao remota ou a URL do binario, sem que essas alteracoes facam parte da versao final entregue.

10. Instrucoes de execucao

1. Abrir o projeto publico do Wokwi pelo link informado na secão 1.

2. Confirmar que o sketch inicial corresponde ao Firmware 1.0.
3. Iniciar a simulacao e abrir o Serial Monitor em 115200 baud.
4. Aguardar tres sessoes completas. Cada sessao possui 5 leituras, com 2 s entre leituras e 48 s entre inicios de sessao.
5. Apos a terceira sessao, acompanhar a consulta ao version.json, a deteccion da versao 2.0, o download do firmware_v2.bin e a mensagem de OTA concluida.
6. Apos o reboot, verificar a identificacao do Firmware 2.0 e as novas informacoes de ordem original, ordem crescente, media, mediana e estado.
7. Observar o LED RGB: azul no FW 1.0; verde no FW 2.0 em NORMAL; vermelho no FW 2.0 em ALERTA.

11. Conclusao

A solucao implementada integra aquisicao simulada de dados, processamento estatistico, temporizacao nao bloqueante, comunicacao HTTPS, versionamento e atualizacao OTA. O teste no Wokwi confirmou o fluxo completo do Firmware 1.0 para o Firmware 2.0 e validou media, ordenacao, mediana, histerese e indicacao visual por LED RGB. O repositorio remoto permanece como ponto de distribuicao do manifesto e do binario usado na atualizacao.

Apendice A - Código-fonte completo do Firmware 1.0

Arquivo correspondente ao firmware inicial usado para executar as três sessões e iniciar a atualização OTA.

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <ArduinoJson.h>
#include <esp_ota_ops.h>

const char* VERSAO = "1.0";
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

const char* MANIFESTO_URL =
    "https://raw.githubusercontent.com/jalbino0/motiva-ota/main/version.json";

const int LED_R = 27;
const int LED_G = 26;
const int LED_B = 25;

const int NUM_LEITURAS = 5;
const unsigned long INTERVALO_LEITURA = 2000;
const unsigned long INTERVALO_SESSAO = 48000;

int leituras[NUM_LEITURAS];
int quantidadeLeituras = 0;
int sessoesConcluidas = 0;

unsigned long inicioSessao = 0;
bool primeiraSessao = true;
bool sessaoAtiva = false;
bool otaConsultada = false;

void conectarWiFi() {
    Serial.println("Conectando ao Wi-Fi...");
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int tentativas = 0;

    while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
        delay(500);
        Serial.print(".");
        tentativas++;
    }

    Serial.println();

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("Wi-Fi conectado!");
        Serial.print("Endereço IP: ");
        Serial.println(WiFi.localIP());
    } else {
        Serial.println("ERRO: sem conexão Wi-Fi.");
    }
}

// Compara versões numéricas no formato maior.menor.
bool lerVersao(const String& texto, int& maior, int& menor) {
    int ponto = texto.indexOf('.');

    if (ponto <= 0 || ponto == texto.length() - 1) {
        return false;
    }

    for (unsigned int i = 0; i < texto.length(); i++) {
        if (static_cast<int>(i) == ponto) {
            continue;
        }

        if (texto[i] < '0' || texto[i] > '9') {
            return false;
        }
    }

    maior = texto.substring(0, ponto).toInt();
    menor = texto.substring(ponto + 1).toInt();

    return true;
}

void mostrarDiagnosticoOTA() {
    Serial.println("===== DIAGNOSTICO DAS PARTICOES =====");

    const esp_partition_t* atual =
        esp_ota_get_running_partition();
    const esp_partition_t* proxima =
```

```

        esp_ota_get_boot_partition();
    if (atual != nullptr) {
        Serial.printf(
            "Particao atual: %s, endereco: 0x%lx\n",
            atual->label,
            static_cast<unsigned long>(atual->address)
        );
    } else {
        Serial.println("ERRO: particao atual nao encontrada.");
    }

    if (proxima != nullptr) {
        Serial.printf(
            "Particao selecionada: %s, endereco: 0x%lx\n",
            proxima->label,
            static_cast<unsigned long>(proxima->address)
        );

        esp_app_desc_t descricao;
        esp_err_t resultado =
            esp_ota_get_partition_description(proxima, &descricao);

        Serial.printf(
            "Verificacao da imagem: %s\n",
            esp_err_to_name(resultado)
        );
    } else {
        Serial.println("ERRO: particao de boot nao encontrada.");
    }

    Serial.println("=====");
}

void consultarOTA() {
    Serial.println();
    Serial.println("===== CONSULTA OTA APOS 3 SESSOES =====");

    if (WiFi.status() != WL_CONNECTED) {
        conectarWiFi();
    }

    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("OTA cancelada: sem conexao Wi-Fi.");
        return;
    }

    String versaoRemota;
    String firmwareURL;

    {
        WiFiClientSecure cliente;

        // Simplificacao para esta demonstracao no simulador:
        // usa HTTPS sem validar o certificado do servidor.
        cliente.setInsecure();

        HTTPClient http;
        http.setTimeout(15000);
        http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);

        if (!http.begin(cliente, MANIFESTO_URL)) {
            Serial.println("ERRO: nao foi possivel iniciar a consulta.");
            return;
        }

        int codigo = http.GET();

        if (codigo != HTTP_CODE_OK) {
            Serial.print("ERRO ao acessar manifesto.Codigo: ");
            Serial.println(codigo);
            http.end();
            return;
        }

        String conteudo = http.getString();
        http.end();

        JsonDocument documento;
        DeserializationError erro =
            deserializeJson(documento, conteudo);

        if (erro) {
            Serial.print("ERRO: manifesto JSON invalido: ");
            Serial.println(erro.c_str());
            return;
        }

        if (!documento["version"].is<const char*>() ||
            !documento["url"].is<const char*>()) {
            Serial.println("ERRO: manifesto sem version ou url validos.");
        }
    }
}

```

```

    return;
}

versaoRemota = documento["version"].as<String>();
firmwareURL = documento["url"].as<String>();
}

int maiorLocal, menorLocal;
int maiorRemoto, menorRemoto;

if (!lerVersao(VERSAO, maiorLocal, menorLocal) ||
    !lerVersao(versaoRemota, maiorRemoto, menorRemoto)) {
    Serial.println("ERRO: use versoes no formato 1.0 ou 2.0.");
    return;
}

Serial.print("Versao instalada: ");
Serial.println(VERSAO);
Serial.print("Versao disponivel: ");
Serial.println(versaoRemota);

bool maisNova =
    maiorRemoto > maiorLocal ||
    (maiorRemoto == maiorLocal && menorRemoto > menorLocal);

if (!maisNova) {
    Serial.println(
        "Firmware ja atualizado: nenhuma versao mais nova."
    );
    return;
}

if (!firmwareURL.startsWith(
    "https://raw.githubusercontent.com/jalbino0/motiva-ota/")
) {
    Serial.println("ERRO: URL fora do repositório esperado.");
    return;
}

Serial.println("Atualizacao disponivel!");
Serial.print("Baixando: ");
Serial.println(firmwareURL);

WiFiClientSecure clienteFirmware;
clienteFirmware.setInsecure();

HTTPUpdate atualizador(15000);
atualizador.rebootOnUpdate(false);
atualizador.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);

t_httpUpdate_return resultado =
    atualizador.update(clienteFirmware, firmwareURL, VERSAO);

if (resultado == HTTP_UPDATE_OK) {
    Serial.println("OTA concluída com sucesso!");
    mostrarDiagnosticoOTA();
    Serial.println("Reiniciando para executar o novo firmware...");
    Serial.flush();
    delay(500);
    ESP.restart();
} else if (resultado == HTTP_UPDATE_NO_UPDATES) {
    Serial.println("Servidor informou que nao ha atualizacao.");
} else {
    Serial.print("ERRO no download ou na gravacao OTA: ");
    Serial.print(atualizador.getLastErrorMessage());
    Serial.print(" - ");
    Serial.println(atualizador.getLastErrorMessage());
}
}

void finalizarSessao() {
    int soma = 0;

    for (int i = 0; i < NUM_LEITURAS; i++) {
        soma += leituras[i];
    }

    float media = soma / static_cast<float>(NUM_LEITURAS);

    Serial.print("Media da sessao: ");
    Serial.print(media, 1);
    Serial.println(" cm");

    sessoesConcluidas++;

    Serial.print("Sessoes concluidas: ");
    Serial.println(sessoesConcluidas);
    Serial.println();

    sessaoAtiva = false;
}

```

```

void setup() {
    Serial.begin(115200);

    pinMode(LED_R, OUTPUT);
    pinMode(LED_G, OUTPUT);
    pinMode(LED_B, OUTPUT);

    digitalWrite(LED_R, LOW);
    digitalWrite(LED_G, LOW);
    digitalWrite(LED_B, HIGH);

    Serial.println("=====");
    Serial.println("MONITORAMENTO DE VEGETACAO - FW 1.0");
    Serial.println("OTA habilitada apos 3 sessoes completas");
    Serial.println("=====");

    conectarWiFi();
}

void loop() {
    unsigned long agora = millis();

    if (!sessaoAtiva &&
        (primeiraSessao ||
         agora - inicioSessao >= INTERVALO_SESSAO)) {

        if (primeiraSessao) {
            inicioSessao = agora;
            primeiraSessao = false;
        } else {
            // Preserva a cadencia normal. Se uma tentativa OTA
            // demorar demais, evita acumular sessoes atrasadas.
            inicioSessao += INTERVALO_SESSAO;

            if (agora - inicioSessao >= INTERVALO_SESSAO) {
                inicioSessao = agora;
            }
        }

        quantidadeLeituras = 0;
        sessaoAtiva = true;

        Serial.println("----- NOVA SESSAO -----");
        Serial.print("Inicio da sessao (ms): ");
        Serial.println(agora);
    }

    if (sessaoAtiva &&
        agora - inicioSessao >=
            quantidadeLeituras * INTERVALO_LEITURA) {

        leituras[quantidadeLeituras] = random(10, 21);

        Serial.print("Leitura ");
        Serial.print(quantidadeLeituras + 1);
        Serial.print(": ");
        Serial.print(leituras[quantidadeLeituras]);
        Serial.println(" cm");

        quantidadeLeituras++;

        if (quantidadeLeituras == NUM_LEITURAS) {
            finalizarSessao();
        }
    }

    if (!sessaoAtiva &&
        sessoesConcluidas >= 3 &&
        !otaConsultada) {

        otaConsultada = true;
        consultarOTA();
    }
}

```

Apendice B - Código-fonte completo do Firmware 2.0

Arquivo final da versão 2.0, com ordenação, mediana, histerese e indicação do estado por LED RGB.

```
#include <Arduino.h>
#include <WiFi.h>

const int LED_R = 27;
const int LED_G = 26;
const int LED_B = 25;

const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

const int NUM_LEITURAS = 5;
const unsigned long INTERVALO_LEITURA = 2000;
const unsigned long INTERVALO_SESSAO = 48000;

int leituras[NUM_LEITURAS];
int quantidadeLeituras = 0;

unsigned long inicioSessao = 0;
bool sessaoAtiva = false;
bool primeiraSessao = true;
bool estadoAlerta = false;

void atualizarLED() {
  digitalWrite(LED_R, estadoAlerta ? HIGH : LOW);
  digitalWrite(LED_G, estadoAlerta ? LOW : HIGH);
  digitalWrite(LED_B, LOW);
}

void conectarWiFi() {
  Serial.println("Conectando ao Wi-Fi...");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int tentativas = 0;

  while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
    delay(500);
    Serial.print(".");
    tentativas++;
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("Wi-Fi conectado com sucesso!");
    Serial.print("Endereço IP: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("ERRO: Não foi possível conectar ao Wi-Fi.");
  }
}

void finalizarSessao() {
  int ordenadas[NUM_LEITURAS];
  int soma = 0;

  Serial.print("Ordem original: ");

  for (int i = 0; i < NUM_LEITURAS; i++) {
    ordenadas[i] = leituras[i];
    soma += leituras[i];

    Serial.print(leituras[i]);
    Serial.print(" ");
  }

  Serial.println();

  // Ordena uma cópia, preservando o vetor original.
  for (int i = 0; i < NUM_LEITURAS - 1; i++) {
    for (int j = 0; j < NUM_LEITURAS - 1 - i; j++) {
      if (ordenadas[j] > ordenadas[j + 1]) {
        int temporario = ordenadas[j];
        ordenadas[j] = ordenadas[j + 1];
        ordenadas[j + 1] = temporario;
      }
    }
  }

  Serial.print("Ordem crescente: ");

  for (int i = 0; i < NUM_LEITURAS; i++) {
    Serial.print(ordenadas[i]);
    Serial.print(" ");
  }
}
```

```

Serial.println();

float media = soma / static_cast<float>(NUM_LEITURAS);
int mediana = ordenadas[2];

Serial.print("Media: ");
Serial.print(media, 1);
Serial.println(" cm");

Serial.print("Mediana: ");
Serial.print(mediana);
Serial.println(" cm");

// Histerese: na faixa intermediaria, preserva o estado.
if (mediana >= 16) {
    estadoAlerta = true;
} else if (mediana <= 14) {
    estadoAlerta = false;
}

atualizarLED();

Serial.print("Estado: ");
Serial.println(estadoAlerta ? "ALERTA" : "NORMAL");
Serial.println("Proxima sessao: 48 segundos apos o inicio desta.");
Serial.println();

sessaoAtiva = false;
}

void setup() {
    Serial.begin(115200);

    pinMode(LED_R, OUTPUT);
    pinMode(LED_G, OUTPUT);
    pinMode(LED_B, OUTPUT);

    atualizarLED();

    Serial.println("=====");
    Serial.println("MONITORAMENTO DE VEGETACAO - FW 2.0");
    Serial.println("=====");

    conectarWiFi();
}

void loop() {
    unsigned long agora = millis();

    if (!sessaoAtiva &&
        (primeiraSessao || agora - inicioSessao >= INTERVALO_SESSAO)) {

        if (primeiraSessao) {
            inicioSessao = agora;
            primeiraSessao = false;
        } else {
            inicioSessao += INTERVALO_SESSAO;
        }

        quantidadeLeituras = 0;
        sessaoAtiva = true;

        Serial.println("----- NOVA SESSAO -----");
        Serial.print("Inicio da sessao (ms): ");
        Serial.println(agora);
    }

    if (sessaoAtiva &&
        agora - inicioSessao >=
            quantidadeLeituras * INTERVALO_LEITURA) {

        leituras[quantidadeLeituras] = random(10, 21);

        Serial.print("Leitura ");
        Serial.print(quantidadeLeituras + 1);
        Serial.print(": ");
        Serial.print(leituras[quantidadeLeituras]);
        Serial.println(" cm");

        quantidadeLeituras++;

        if (quantidadeLeituras == NUM_LEITURAS) {
            finalizarSessao();
        }
    }
}

```

Apendice C - Manifesto e arquivos da atualizacao

C.1 version.json

```
{  
  "version": "2.0",  
  "url":  
    "https://raw.githubusercontent.com/jalbino0/motiva-ota/main/firmware_v2.bin"  
}
```

C.2 Binario utilizado

Arquivo: firmware_v2.bin

URL direta: https://raw.githubusercontent.com/jalbino0/motiva-ota/main/firmware_v2.bin

C.3 Repositorio remoto

<https://github.com/jalbino0/motiva-ota>

C.4 Projeto Wokwi

<https://wokwi.com/projects/475891405739849729>